

Batching, Training, Evaluation, and Future Work

Govind

July 17, 2025

1 Batching

Small batch sizes (e.g. 2) yield significant speedups despite mesh irregularity. We implement batching by:

1. Concatenating per-graph vertex arrays into one $[\sum N_v, 3]$ tensor.
2. Offsetting edge-index pairs by cumulative vertex counts to avoid cross-graph edges.
3. Feeding the combined disjoint graph into shared GNN modules.

Learned the hard way: to swap axes you must use `.T` (transpose), not `.view` or `.reshape`, since those only reinterpret strides without permuting memory. Trained models can be visualized using the `visualize_model_outputs.ipynb` notebook.

2 Attention Module Training

The `VertexAttentionModule` is pre-trained on per-vertex binary masks with

$$\mathcal{L}_{\text{attn}} = \text{BCEWithLogitsLoss}(\text{logits}, m),$$

where mask fraction $p \approx 0.27$ yields positive weight $(1 - p)/p \approx 2.7$. Over thousands of epochs this yields very low BCE losses and high accuracy and F1 scores on the positive class.

3 Displacement Module Training

We train the `JointDisplacementModule` using symmetric Chamfer distance:

$$\mathcal{L}_{\text{disp}} = \frac{1}{V} \sum_v \min_k \|q_v - t_k\| + \frac{1}{K} \sum_k \min_v \|q_v - t_k\|,$$

where q_v are displaced vertices and t_k ground-truth joints. To assess localization we compute **Percentage of Correct Keypoints (PCK)**:

- **PCK@ τ** : fraction of predicted points within distance τ of any GT joint.
- Sweep τ over $\{0.005, 0.01, \dots\}$ to get a PCK curve.
- The **Area Under Curve (AUC)** summarizes overall accuracy: higher AUC means better localization even at tight tolerances.

4 Training Details

Our scripts now use a robust checkpoint system:

- **Rolling checkpoints**: save `latest.pt` each epoch (configurable via `save-every`).
- **Crash recovery**: on OOM or unexpected errors, reload `latest.pt` and resume.
- **Best-model tracking**:
 - Attention head: save checkpoint with highest F1 and lowest BCE loss.
 - Displacement head: save checkpoint with lowest Chamfer loss and highest PCK AUC.
- **Mixed-precision**: use `torch.cuda.amp` and a `GradScaler` to automatically cast between FP32/FP16, halving memory use and eliminating most OOM crashes.

Each epoch runs in 1 min; full 5000-epoch jobs take multiple days. Both attention and displacement trainings are ongoing.

5 Clustering Bandwidth Training

The paper fine-tunes attention, displacement, and bandwidth h jointly via combined losses, but during debugging we discovered:

- Mode extraction (an `argmax`) is non-differentiable—blocks gradients.
- Joint updates of h with other weights produce unstable, spiky losses.

Possible solutions:

1. **Hyperparameter sweep**: grid-search h by validating Chamfer + PCK/AUC.
2. **Staged training**: freeze attention & displacement, then backpropagate only through h on cluster-point sets (not mode outputs).

6 Evaluation Metrics

BCE Loss: per-vertex binary classification for attention masks.

Chamfer Loss: symmetric distance for displacement heads.

PCK Curve: localization performance at multiple thresholds.

AUC: area under the PCK curve.

Precision/Recall: for the positive class in attention.

7 Next Steps

- Scheduler script to launch hyperparameter sweeps (attention + displacement), select best models by validation metrics.
- Full clustering-head training via staged/freeze or differentiable approximations of mode extraction.
- Symmetry-based post-processing: mirror predicted joints across the mesh's symmetry plane.
- Bone-prediction stage: handle variable-length joint sets versus ground-truth inputs.